

01 字符串与数据结构基础

专项目标：把原始输入清洗成可处理的数据结构，再完成访问、修改、遍历、转换和输出。

来源优先级：期末范围、EXAM_PREP、老师课堂和实验报告、已有理论笔记。

总主线

```
input 得到字符串
→ strip 清理
→ split 拆分
→ int/float 转换
→ 保存为 list/tuple/dict/set
→ 遍历与处理
→ join 或格式化输出
```

考试里很多题看起来不同，本质都在问：

```
原始数据是什么？
拆开后每一项是什么？
要保存成什么结构？
循环每轮拿到什么？
最后怎么输出？
```

字符串输入处理【必须闭卷会写】

1. 它解决什么问题

用户输入、文件读取、CSV 一行文本，最开始通常都是字符串。程序要先清洗、拆分、转换，才能计算。

2. 最小语法

```
text = input().strip()
parts = text.split()
numbers = [int(part) for part in parts]
```

3. 执行过程

```
" 80 90 70 "  
→ strip() 得到 "80 90 70"  
→ split() 得到 ["80", "90", "70"]  
→ int() 得到 [80, 90, 70]
```

4. 普通循环写法

```
raw = "80 90 70"  
parts = raw.split()  
  
scores = []  
for part in parts:  
    scores.append(int(part))  
  
print(scores)
```

输出：

```
[80, 90, 70]
```

5. 简化写法

```
scores = [int(part) for part in "80 90 70".split()]  
print(scores)
```

6. 常见错误

```
scores = "80 90 70".split()  
print(sum(scores)) # TypeError, scores 里还是字符串
```

正确：

```
scores = [int(part) for part in "80 90 70".split()]  
print(sum(scores))
```

7. 考试可能怎么出

给一行成绩字符串，要求求平均分、最高分、及格人数。

字符串方法速查【必须闭卷会写】

方法	作用	高频提醒
s[i]	索引取字符	下标从 0 开始
s[a:b]	切片	左闭右开
s[::-1]	反转	回文题常用
strip()	去两端空白	不改原字符串
lstrip() / rstrip()	去左/右空白	清理输入
split()	字符串转列表	默认按空白拆
join()	字符串序列转字符串	成员必须是字符串
replace(old, new)	替换	返回新字符串
find()	找位置	找不到返回 -1
index()	找位置	找不到报错
in / not in	判断包含	比 find() != -1 更直观
isdigit()	是否全数字	"80" 为 True
isalpha()	是否全字母	中文也算字母
isalnum()	字母或数字	空格、标点不算
lower() / upper()	大小写转换	文件后缀判断常用
startswith() / endswith()	开头/结尾判断	文件格式检测常用

重点记忆：

```
split: 字符串 → 列表
join: 字符串序列 → 字符串
join 的成员必须都是字符串
字符串不可变，字符串方法通常返回新字符串
```

例 1： "Tom,80" 转姓名和成绩

```
line = "Tom,80"
name, score_text = line.split(",")
score = int(score_text)

print(name)
print(score)
```

输出：

```
Tom
80
```

例 2： "Tom:80,Jerry:92" 转成绩字典

```
text = "Tom:80,Jerry:92"
items = text.split(",")

scores = {}
for item in items:
    name, score_text = item.split(":")
    scores[name] = int(score_text)

print(scores)
```

输出：

```
{'Tom': 80, 'Jerry': 92}
```

例 3：数字列表转逗号分隔字符串

```
nums = [80, 90, 70]
parts = []

for num in nums:
    parts.append(str(num))

result = ",".join(parts)
print(result)
```

输出：

```
80,90,70
```

例 4：使用 `endswith()` 筛选头像文件

```
files = ["me.jpg", "data.txt", "avatar.PNG", "photo.jpeg"]

for filename in files:
```

```
lower_name = filename.lower()
if lower_name.endswith(".jpg") or lower_name.endswith(".png") or
lower_name.endswith(".jpeg"):
    print(filename)
```

四种数据结构总对比【必须闭卷会写】

结构	是否有序	是否允许重复	是否可修改	如何访问	主要应用	能否作为字典键
list	有序	允许	可修改	索引、切片	多条同类数据	不能
tuple	有序	允许	不可修改	索引、切片	固定数据、多返回值	通常可以
dict	保持插入顺序	key 不重复	可修改	key	按名字/编号查值	不能整体做键
set	不应依赖顺序	不允许重复	可修改	不能索引	去重、交并差	不能

注意：集合不支持通过索引访问，不应依赖其输出顺序。

列表 list【必须闭卷会写】

1. 它解决什么问题

列表保存“一批有顺序的数据”，例如多个成绩、多个学生字典、多个文件名。

2. 最小语法

```
scores = [80, 90, 70]
print(scores[0])
print(scores[-1])
print(scores[1:3])
print(scores[::-1])
```

3. 执行过程

```
scores[0] 取第 1 个  
scores[-1] 取最后 1 个  
scores[1:3] 取索引 1、2  
scores[::-1] 反向切片
```

4. 增删改查

```
nums = [1, 2, 3]  
  
nums.append(4)          # [1, 2, 3, 4]  
nums.extend([5, 6])     # [1, 2, 3, 4, 5, 6]  
nums.insert(1, 99)      # [1, 99, 2, 3, 4, 5, 6]  
  
nums.remove(99)         # 按值删除, 不存在 ValueError  
last = nums.pop()       # 删除并返回最后一个  
del nums[0]             # 按索引删除
```

`append([4, 5])` 和 `extend([4, 5])` 的区别:

```
a = [1, 2]  
a.append([3, 4])  
print(a) # [1, 2, [3, 4]]  
  
b = [1, 2]  
b.extend([3, 4])  
print(b) # [1, 2, 3, 4]
```

5. 排序和统计

```
scores = [80, 60, 90]  
  
scores.sort()  
print(scores) # [60, 80, 90]  
  
new_scores = sorted(scores, reverse=True)  
print(new_scores) # [90, 80, 60]  
  
print(len(scores))  
print(sum(scores))  
print(max(scores))  
print(min(scores))
```

重点：

```
sort 修改原列表并返回 None
sorted 返回新列表
reverse 只是反转当前顺序，不是排序
```

6. 列表陷阱【必须会读会改】

```
a = [1, 2, 3]
b = a
b.append(4)
print(a) # [1, 2, 3, 4]
```

`b = a` 是共享引用，不是复制。

浅拷贝：

```
a = [1, 2, 3]
b = a.copy()
b.append(4)
print(a) # [1, 2, 3]
```

嵌套列表的浅拷贝问题：

```
a = [[1, 2], [3, 4]]
b = a.copy()
b[0].append(99)
print(a) # [[1, 2, 99], [3, 4]]
```

深拷贝了解即可，遇到嵌套可变对象才需要。

遍历原列表时删除元素容易漏删：

```
nums = [1, 2, 3, 4]
result = []
for num in nums:
    if num % 2 != 0:
        result.append(num)
print(result)
```

元组 tuple 【必须闭卷会写】

1. 它解决什么问题

元组保存固定数据，也常用于函数返回多个值。

2. 最小语法

```
point = (3, 4)
single = (1,)
print(point[0])
```

单元素元组必须有逗号：

```
print(type((1)))    # int
print(type((1,)))   # tuple
```

3. 基础解包

```
name, score = ("Tom", 80)
print(name, score)

a, b = 1, 2
a, b = b, a
print(a, b)
```

_ 占位符和 * 剩余解包：

```
name, _, city = ("Tom", 18, "Nanchang")
first, *middle, last = [1, 2, 3, 4, 5]
```

4. 函数多返回值本质

```
def get_min_max(scores):
    return min(scores), max(scores)

lowest, highest = get_min_max([80, 90, 70])
print(lowest, highest)
```

字典 dict 【必须闭卷会写】

1. 它解决什么问题

字典保存“名字 -> 值”的映射，例如姓名到成绩、商品名到价格。

2. 最小语法

```
student = {"name": "Tom", "score": 80}
print(student["name"])
print(student.get("age", "未知"))
```

`d[key]` 和 `d.get()`：

`d[key]`: `key` 不存在会报错
`d.get(key, 默认值)`: `key` 不存在返回默认值

3. 增删改查

```
student = {}
student["name"] = "Tom"
student["score"] = 80
student["score"] = 90

student.pop("score")
student["score"] = 88
del student["score"]
student.clear()
```

4. 遍历

```
student = {"name": "Tom", "score": 80}

for key in student:
    print(key)

for value in student.values():
    print(value)

for key, value in student.items():
    print(key, value)
```

in 默认判断键：

```
print("name" in student)
```

5. 字典计数模板

```
text = "abc1ab"
counts = {}

for ch in text:
    counts[ch] = counts.get(ch, 0) + 1

print(counts)
```

输出：

```
{'a': 2, 'b': 2, 'c': 1, '1': 1}
```

6. 嵌套字典和字典列表

```
students = [
    {"name": "Tom", "score": 80},
    {"name": "Jerry", "score": 92},
]

for student in students:
    print(student["name"], student["score"])
```

字典键必须可哈希：列表不能作为字典键，元组通常可以。

集合 set 【必须闭卷会写】

1. 它解决什么问题

集合用于去重、判断是否存在、求交集并集差集。

2. 最小语法

```
nums = {1, 2, 2, 3}
print(nums)  # {1, 2, 3}
```

```
empty = set()
```

空集合必须用 `set()`，`{}` 是空字典。

3. 增删查和集合运算

```
s = {1, 2}
s.add(3)
s.update([3, 4, 5])
s.remove(1)      # 不存在会报错
s.discard(99)    # 不存在不报错

a = {1, 2, 3}
b = {3, 4}
print(a | b)     # 并集
print(a & b)     # 交集
print(a - b)     # 差集
print(a ^ b)     # 对称差
```

4. 去重但保留原顺序

`list(set(data))` 可以去重，但不保证保留原顺序。

普通循环写法：

```
data = [1, 2, 1, 3, 2]
result = []
seen = set()

for item in data:
    if item not in seen:
        result.append(item)
        seen.add(item)

print(result)
```

输出：

```
[1, 2, 3]
```

数据结构互转【必须闭卷会写】

字符串 ↔ 列表

```
parts = "a,b,c".split(",")
text = ",".join(parts)
```

列表 ↔ 元组

```
nums = [1, 2, 3]
t = tuple(nums)
nums2 = list(t)
```

列表/元组 ↔ 集合

```
nums = [1, 2, 2, 3]
unique = set(nums)
back = list(unique)
```

键值对列表 → 字典

```
pairs = [("Tom", 80), ("Jerry", 92)]
scores = dict(pairs)
```

`dict([1, 2, 3])` 报错，因为每一项不是 (key, value)。

字典 → keys/values/items 列表

```
d = {"Tom": 80, "Jerry": 92}
print(list(d))           # 只得到键
print(list(d.keys()))    # 键
print(list(d.values()))  # 值
print(list(d.items()))   # 键值对
```

`dict(zip(keys, values))`

```
names = ["Tom", "Jerry"]
scores = [80, 92]
score_dict = dict(zip(names, scores))
print(score_dict)
```

一页方法速查

类型	高频方法
str	strip、split、join、replace、find、lower、endswith
list	append、extend、insert、remove、pop、sort、copy
tuple	count、index、解包
dict	get、pop、keys、values、items
set	add、update、remove、discard、`

10 个易错点

1. input() 得到的是字符串。
2. split() 后每一项仍然是字符串。
3. join() 的成员必须都是字符串。
4. 字符串方法返回新字符串，不原地修改。
5. find() 找不到返回 -1，index() 找不到报错。
6. append([1, 2]) 会把整个列表作为一个元素。
7. sort() 返回 None。
8. {} 是空字典，不是空集合。
9. 字典默认遍历键。
10. 集合去重不保证保留原顺序。

8 个口头自测问题

1. split() 和 join() 的方向分别是什么？
2. 为什么 ", ".join([1, 2, 3]) 会报错？
3. append 和 extend 有什么区别？
4. sort 和 sorted 有什么区别？
5. 单元素元组为什么要写 (1,)？
6. d[key] 和 d.get(key) 的区别是什么？
7. remove 和 discard 的区别是什么？
8. list(dict_data) 得到什么？

5 个最小闭卷练习

题 1

把 "80 90 75" 转成整数列表。

题 2

把 "Tom,80" 拆成姓名和整数成绩。

题 3

统计 "abc1ab" 中每个字符出现次数。

题 4

对 [1, 2, 1, 3, 2] 去重并保留原顺序。

题 5

把 names = ["Tom", "Jerry"] 和 scores = [80, 92] 转成字典。

答案区

答 1

```
scores = []
for part in "80 90 75".split():
    scores.append(int(part))
print(scores)
```

答 2

```
name, score_text = "Tom,80".split(",")
score = int(score_text)
print(name, score)
```

答 3

```
counts = {}
for ch in "abc1ab":
    counts[ch] = counts.get(ch, 0) + 1
print(counts)
```

答 4

```
nums = [1, 2, 1, 3, 2]
result = []
seen = set()

for num in nums:
    if num not in seen:
        result.append(num)
        seen.add(num)

print(result)
```

答 5

```
names = ["Tom", "Jerry"]
scores = [80, 92]
result = dict(zip(names, scores))
print(result)
```

[继续学习函数综合](#)